

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Naveen Kumar S	Reg. No.:	192421030
Section / Batch:	B Tech IT	Date:	18.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I Naveen Kumar S - 192421030 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Naveen Kumar S

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				

Total Marks (100):	
---------------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



SIMATS Online Programming Lab
DAA PROGRAMMING


NAVEEN KUMAR S
192421030RC

Monitoring system processes event logs that must be sorted and searched efficiently
100 marks
1/1 answered

and conquer techniques. Analyze complexity and justify the design.

Your Code

```

1 # Event Log Monitoring System (Sorting + Binary Search)
2 # All input is inbuilt within the code for immediate execution.
3
4 def merge_sort(arr):
5     """Implementation of Merge Sort (Divide and Conquer)"""
6     if len(arr) <= 1:
7         return arr
8
9     mid = len(arr) // 2
10    left = merge_sort(arr[:mid])
11    right = merge_sort(arr[mid:])
12
13    return merge(left, right)
14
15
16 def merge(left, right):
17     """Helper function to merge two sorted arrays"""
18     result = []
19     i = j = 0
20
21     while i < len(left) and j < len(right):

```

Input

stdin input

Output

```

---
Event Log Monitoring System
---
Raw Log Data: [1024, 567, 8923,
4321, 11, 999, 5543, 2345, 778,
3456]
Searching for Event Timestamp:
4321

```

This problem is an **Inventory Replenishment Optimization using Dynamic Programming (DP)**. The goal is to decide **how much inventory to order in each period** so that the total holding, ordering, and shortage cost is minimized.

1. Problem Formulation

Suppose a retail chain plans inventory for T periods.

For each period t :

- I_t = inventory available at the beginning of period t
- Q_t = quantity ordered in period t
- D_t = demand in period t

- H = holding cost per unit
- S = shortage cost per unit
- C = ordering cost per unit
- M = maximum storage capacity

Inventory after satisfying demand is:

$$I_{t+1} = I_t + Q_t - D_t$$

If demand is greater than available inventory, a shortage occurs.

2. Dynamic Programming State

The DP state can be defined as:

$$DP(t, I)$$

where:

- t = current time period
- I = inventory level at the beginning of that period

$DP(t, I)$ represents:

Minimum total cost from period t to the end when the current inventory is I .

3. Cost Function

For an order quantity Q , inventory before demand is:

$$I + Q$$

After demand:

$$I' = I + Q - D_t$$

If $I' \geq 0$, we have leftover inventory and therefore a holding cost:

$$\text{HoldingCost} = H(I')$$

If $I' < 0$, there is a shortage:

$$\text{ShortageCost} = S(-I')$$

The total cost for the decision is therefore:

$$\text{Cost} = CQ + H\max(I', 0) + S\max(-I', 0)$$

4. Recurrence Relation

The main DP recurrence is:

$$DP(t, I) = Q\min[CQ + H\max(I + Q - Dt, 0) + S\max(Dt - I - Q, 0) + DP(t+1, I + Q - Dt)]$$

This means:

For every possible order quantity Q , calculate the current cost plus the optimal future cost, and choose the order quantity that gives the minimum total cost.

Base case

After the final period:

$$DP(T+1, I) = 0$$

if no terminal inventory penalty is considered.

5. Algorithm

Algorithm: Inventory Replenishment using Dynamic Programming

Input:

T = number of periods

Initial inventory I_0

Demand $D[1 \dots T]$

Holding cost H

Shortage cost S

Ordering cost C

Maximum storage capacity M

Create DP table $DP[t][I]$

Set:

$DP[T+1][I] = 0$ for all possible inventory levels

For $t = T$ down to 1:

For every possible inventory level I :

$DP[t][I] = \text{infinity}$

For every feasible order quantity Q :

$\text{Available} = I + Q$

$\text{Remaining} = \text{Available} - D[t]$

$\text{HoldingCost} = H \times \max(\text{Remaining}, 0)$

$\text{ShortageCost} = S \times \max(-\text{Remaining}, 0)$

$\text{OrderingCost} = C \times Q$

$\text{TotalCost} =$

OrderingCost

$+ \text{HoldingCost}$

$+ \text{ShortageCost}$

$+ DP[t+1][\text{Remaining}]$

If $\text{TotalCost} < DP[t][I]$:

$DP[t][I] = \text{TotalCost}$

Store Q as the optimal decision

Output:

Optimal order quantity for every period

Minimum total cost

Inventory levels

Total holding cost

Total shortage cost

Total ordering cost

